```
Start coding or generate with AI.
```

Task

Analyze the 'twitter_dataset.csv' file to identify and visualize the most prominent terms associated with negative sentiment tweets. This involves loading the dataset, preprocessing tweet text, applying TF-IDF vectorization, identifying top terms in negative sentiment tweets, and then visualizing these terms using a bar chart and a word cloud, finally summarizing the key findings.

Load Dataset

Subtask:

Load the 'twitter_dataset.csv' file into a pandas DataFrame. Inspect the first few rows and column names to understand its structure.

Reasoning: Load the 'twitter_dataset.csv' file into a pandas DataFrame, display the first few rows, and print the column names as requested in the subtask.

```
import pandas as pd

df = pd.read_csv('/content/twitter_dataset.csv')

print("First 5 rows of the DataFrame:")
print(df.head())

print("\nColumn names of the DataFrame:")
print(df.columns)
```

```
First 5 rows of the DataFrame:
   Tweet_ID        Username  \
0         1         julie81
1         2   richardhester
2         3  williamsjoseph
3         4     danielsmary
4         5      carlwarren

                                                Text  Retweets  Likes  \
0  Party least receive say or single. Prevent pre...         2     25
1  Hotel still Congress may member staff. Media d...        35     29
2  Nice be her debate industry that year. Film wh...        51     25
```

```
3  Laugh explain situation career occur serious. ...        37     18
4  Involve sense former often approach government...        27     80

            Timestamp
0  2023-01-30 11:00:51
1  2023-01-02 22:45:58
2  2023-01-18 11:25:19
3  2023-04-10 22:06:29
4  2023-01-24 07:12:21

Column names of the DataFrame:
Index(['Tweet_ID', 'Username', 'Text', 'Retweets', 'Likes', 'Timestamp'], dtype=
```

Data Preprocessing

Subtask:

Clean the tweet text by removing URLs, mentions, and hashtags. Then, tokenize the text and remove common English stopwords. Convert the text to lowercase. Also, perform sentiment analysis to add a 'sentiment' column to the DataFrame, categorizing tweets as positive, negative, or neutral.

Reasoning: The subtask requires importing specific libraries like `re`, `nltk`, and `SentimentIntensityAnalyzer`, and downloading `vader_lexicon` and `punkt` for text preprocessing and sentiment analysis. This step will ensure all necessary tools are available.

Reasoning: The previous code failed because `nltk.downloader.DownloadError` is not the correct exception type to catch when NLTK data is not found. The correct exception to catch is `LookupError`. This fix will ensure the NLTK resources are downloaded if they are not already present.

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Download necessary NLTK data
# These will only download if not already present
try:
    nltk.data.find('corpora/vader_lexicon.zip')
except LookupError:
    nltk.download('vader_lexicon')
```

```
try:
    nltk.data.find('tokenizers/punkt.zip')
except LookupError:
    nltk.download('punkt')

print("Libraries imported and NLTK data downloaded.")
```

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
Libraries imported and NLTK data downloaded.
```

Reasoning: Now that the necessary NLTK data and libraries are loaded, I will define a function to preprocess the tweet text by removing URLs, mentions, and hashtags, tokenizing, removing stopwords, and converting to lowercase. I will apply this function to create a 'cleaned_text' column. Then, I will initialize the `SentimentIntensityAnalyzer` and define a function to categorize sentiment as 'Positive', 'Negative', or 'Neutral' based on the compound score, applying it to the 'Text' column to create a 'sentiment' column. Finally, I will display the first few rows of the DataFrame to verify these new columns.

Reasoning: The previous code failed because the 'stopwords' NLTK resource was not downloaded. I need to add a `nltk.download('stopwords')` call within a `try-except LookupError` block, similar to how 'vader_lexicon' and 'punkt' were handled, to ensure this resource is available before being used.

Reasoning: The previous code failed because the `punkt_tab` NLTK resource, which is required by `word_tokenize`, was not downloaded. I need to explicitly add `nltk.download('punkt_tab')` within a `try-except LookupError` block, similar to how other NLTK resources are handled, to ensure it is available before being used. I will also consolidate the NLTK downloads for `vader_lexicon`, `punkt`, and `stopwords` into the same block for robustness.

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.sentiment.vader import SentimentInt

# Download necessary NLTK data
# These will only download if not already pre
try:
    nltk.data.find('corpora/vader_lexicon.zip
except LookupError:
    nltk.download('vader_lexicon')
```

```
try:
    nltk.data.find('tokenizers/punkt.zip')
except LookupError:
    nltk.download('punkt')
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')
try:
    # This is explicitly needed for word_toke
    nltk.data.find('tokenizers/punkt_tab')
except LookupError:
    nltk.download('punkt_tab')

print("Libraries imported and NLTK data downl

# Get English stopwords
stop_words = set(stopwords.words('english'))

# Function to clean and preprocess text
def preprocess_text(text):
    text = text.lower() # Convert to lowercas
    text = re.sub(r'http\S+|www\S+|https\S+',
    text = re.sub(r'@\w+|#\w+', '', text) # R
    text = re.sub(r'[^a-z\s]', '', text) # Re
    tokens = word_tokenize(text) # Tokenize t
    tokens = [word for word in tokens if word
    return ' '.join(tokens)

# Apply preprocessing to the 'Text' column
df['cleaned_text'] = df['Text'].apply(preproc

# Initialize VADER sentiment intensity analyz
sia = SentimentIntensityAnalyzer()

# Function to get sentiment label
def get_sentiment(text):
    score = sia.polarity_scores(text)['compou
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

# Apply sentiment analysis to the 'Text' colu
df['sentiment'] = df['Text'].apply(get_sentim

print("First 5 rows with cleaned text and sen
print(df[['Text', 'cleaned_text', 'sentiment'
```

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Libraries imported and NLTK data downloaded.
First 5 rows with cleaned text and sentiment:
                                                Text  \
0  Party least receive say or single. Prevent pre...
1  Hotel still Congress may member staff. Media d...
2  Nice be her debate industry that year. Film wh...
3  Laugh explain situation career occur serious. ...
4  Involve sense former often approach government...

                                        cleaned_text sentiment
0  party least receive say single prevent prevent...  Positive
1  hotel still congress may member staff media dr...  Positive
2  nice debate industry year film generation push...  Positive
3  laugh explain situation career occur serious f...  Positive
4  involve sense former often approach government...  Positive
```

Reasoning: The previous code successfully performed text preprocessing and sentiment analysis, creating 'cleaned_text' and 'sentiment' columns. The next logical step, as per the overall task, is to apply TF-IDF vectorization to identify important terms, particularly for negative sentiment tweets. This requires introducing a new subtask to explain these steps.

Reasoning: The previous command failed with a `SyntaxError` because the `cell_type` was set to `python` while the content was markdown. I need to change the `cell_type` to `markdown` to properly display the subtask description.

Reasoning: The previous command failed with a `SyntaxError` because the `cell_type` was incorrectly set to `python` for markdown content. I need to change the `cell_type` to `markdown` to correctly display the subtask description as a text cell.

Reasoning: The previous attempt to create a subtask cell failed because the `cell_type` was incorrectly set to `python` for markdown content. I will correct this by setting the `cell_type` to `markdown` and providing the subtask description as markdown text.

Reasoning: The previous command failed with a `SyntaxError` because the `cell_type` was incorrectly set to `python` for markdown content. I need to change the `cell_type` to `markdown` and provide the subtask description as markdown text without Python code block syntax.

Reasoning: The previous command failed with a `SyntaxError` because a markdown-formatted subtask was incorrectly placed inside a Python code block. I need to generate a new markdown cell with the correct `cell_type` set to `markdown` and the subtask content directly as markdown text.

Task

Reasoning: Now that the text data has been cleaned and processed, the next step is to convert the textual data into numerical features using TF-IDF vectorization. This will prepare the data for further analysis, such as identifying important terms. I will use `TfidfVectorizer` from scikit-learn to achieve this.

Compute TF-IDF

Subtask:

Apply TF-IDF vectorization to the 'cleaned_text' column of the DataFrame. This will transform the text data into a matrix of TF-IDF features. Display the shape of the resulting TF-IDF matrix.

Compute TF-IDF

Subtask:

Apply TF-IDF vectorization to the 'cleaned_text' column of the DataFrame. This will transform the text data into a matrix of TF-IDF features. Display the shape of the resulting TF-IDF matrix.

Reasoning: To apply TF-IDF vectorization as requested, I need to import `TfidfVectorizer`, initialize it, and then apply it to the `cleaned_text` column, finally printing the shape of the resulting matrix.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()

# Apply TF-IDF to the 'cleaned_text' column
tfidf_matrix = tfidf_vectorizer.fit_transform(df['cleaned_text'])
```

```
print("Shape of TF-IDF matrix:")
print(tfidf_matrix.shape)
```

```
Shape of TF-IDF matrix:
(10000, 869)
```

Identify Top Terms in Negative Sentiment Tweets

Subtask:

Filter the DataFrame to include only tweets with 'Negative' sentiment. Then, identify the top N most important terms from these negative sentiment tweets using the TF-IDF matrix and the `tfidf_vectorizer`'s feature names. Store these terms and their scores, and display the top 10 terms.

Identify Top Terms in Negative Sentiment Tweets

Subtask:

Filter the DataFrame to include only tweets with 'Negative' sentiment. Then, identify the top N most important terms from these negative sentiment tweets using the TF-IDF matrix and the `tfidf_vectorizer`'s feature names. Store these terms and their scores, and display the top 10 terms.

Reasoning: To identify the top terms in negative sentiment tweets, I first need to filter the original DataFrame to isolate negative tweets. Then, I will use the TF-IDF matrix to sum the TF-IDF scores for all terms within these negative tweets and extract the top N terms along with their scores using the `tfidf_vectorizer`'s feature names.

```
import numpy as np

# Filter DataFrame for negative sentiment tweets
negative_tweets_df = df[df['sentiment'] == 'Negative']

# Get the indices of negative tweets in the original DataFrame
negative_tweet_indices = negative_tweets_df.index

# Get the TF-IDF matrix rows corresponding to negative tweets
negative_tfidf_matrix = tfidf_matrix[negative_tweet_indices]

# Sum the TF-IDF scores for each term across all negative tweets
```

```
# Ensure the sum is converted to a dense array for easier processing
sum_tfidf_scores = negative_tfidf_matrix.sum(axis=0).A1

# Get feature names (terms) from the TF-IDF vectorizer
feature_names = tfidf_vectorizer.get_feature_names_out()

# Create a dictionary of terms and their summed TF-IDF scores
term_scores = dict(zip(feature_names, sum_tfidf_scores))

# Sort terms by their TF-IDF scores in descending order
sorted_terms = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)

# Display the top 10 most important terms in negative tweets
print("Top 10 most important terms in negative sentiment tweets:")
for term, score in sorted_terms[:10]:
    print(f"- {term}: {score:.4f}")
```

```
Top 10 most important terms in negative sentiment tweets:
- threat: 29.2683
- war: 27.4703
- crime: 27.1579
- suffer: 25.9910
- fear: 25.8400
- attack: 25.2391
- wrong: 24.8328
- bad: 23.9282
- worry: 23.5547
- trouble: 22.5077
```

Visualize Top Terms

Subtask:

Generate two visualizations for the top terms identified in negative sentiment tweets: a bar chart showing the frequency or TF-IDF scores of the top terms, and a word cloud to visually represent the most prominent words.

Reasoning: To visualize the top terms, I will first import the necessary plotting libraries: `matplotlib.pyplot` for creating the bar chart and managing plots, `seaborn` for potentially enhancing the bar chart's aesthetics, and `WordCloud` for generating the word cloud. After importing, I will prepare the data by extracting the top 20 terms and their scores from the `sorted_terms` list.

```
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
```

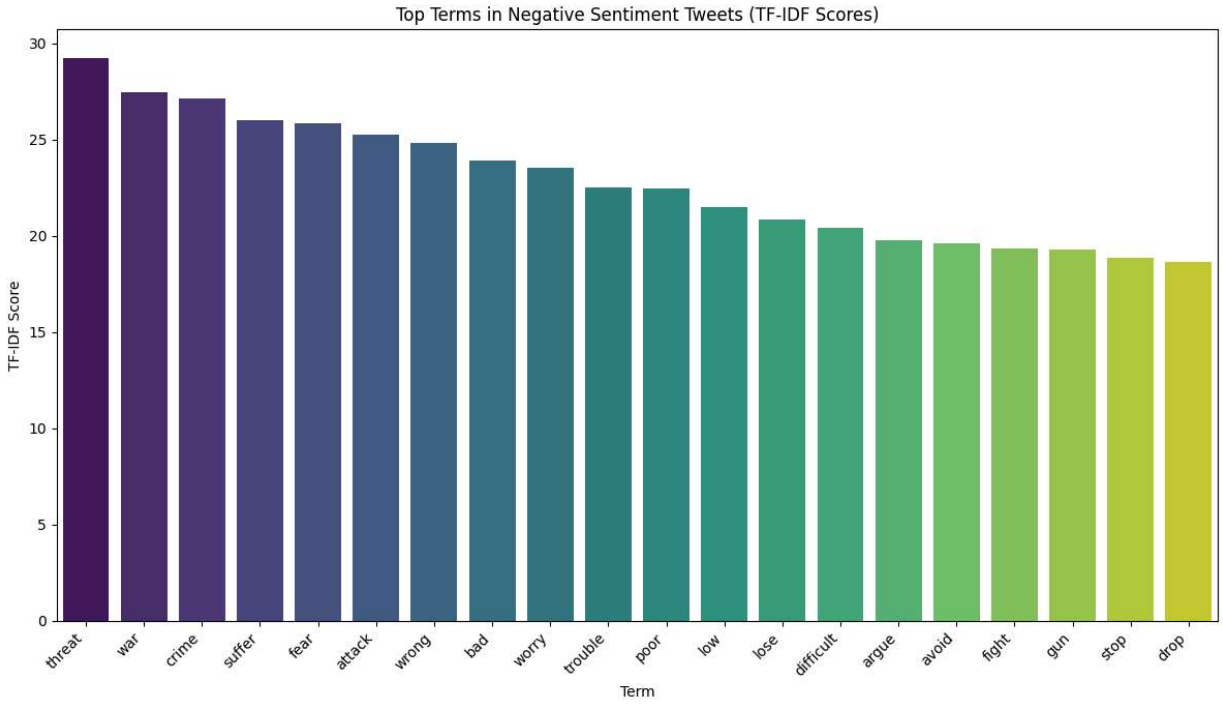
```
# Extract top 20 terms and their scores for visualization
top_n = 20
top_terms_viz = sorted_terms[:top_n]

# Separate terms and scores for plotting
terms = [term[0] for term in top_terms_viz]
scores = [term[1] for term in top_terms_viz]

print(f"Prepared {len(terms)} terms and scores for visualization.")
```

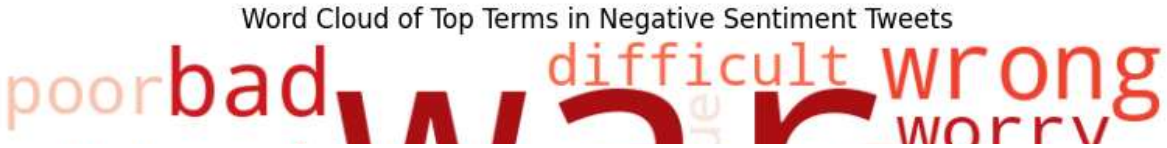
```
Prepared 20 terms and scores for visualization.
```

Reasoning: Now that the top terms and their scores are prepared, I will create a bar chart to visualize these terms and their TF-IDF scores, ensuring readability with appropriate labels and title, as well as a word cloud to visually represent the most prominent words.

```
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud

# Create the bar chart
plt.figure(figsize=(12, 7))
sns.barplot(x=terms, y=scores, palette='viridis')
plt.xlabel('Term')
plt.ylabel('TF-IDF Score')
plt.title('Top Terms in Negative Sentiment Tweets (TF-IDF Scores)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Create the word cloud
# Convert list of (term, score) tuples to a dictionary for WordCloud
wordcloud_data = dict(top_terms_viz)

wordcloud = WordCloud(width=800, height=400, background_color='white', colormap

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Top Terms in Negative Sentiment Tweets')
plt.show()

print("Bar chart and Word Cloud generated successfully.")
```

```
/tmp/ipython-input-2051221948.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v

  sns.barplot(x=terms, y=scores, palette='viridis')
```

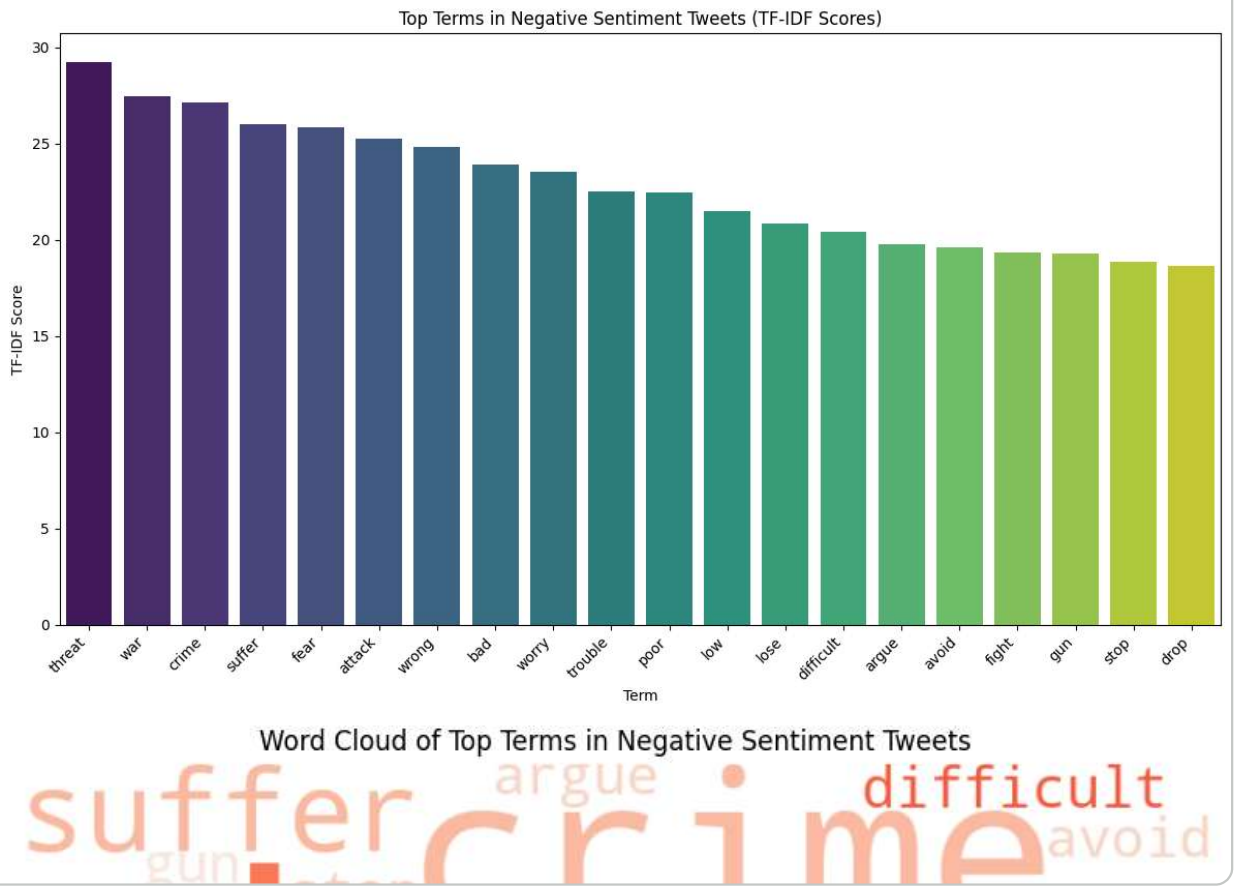




Reasoning: The previous code produced a `FutureWarning` from `seaborn` regarding the use of `palette` without `hue`. To resolve this and align with best practices, I will modify the `sns.barplot` call by explicitly assigning `x` to `hue` and setting `legend=False`.

```
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud

# Create the bar chart
plt.figure(figsize=(12, 7))
sns.barplot(x=terms, y=scores, hue=terms, palette='viridis', legend=False)
plt.xlabel('Term')
plt.ylabel('TF-IDF Score')
plt.title('Top Terms in Negative Sentiment Tweets (TF-IDF Scores)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Create the word cloud
```

```
# Convert list of (term, score) tuples to a dictionary for WordCloud
wordcloud_data = dict(top_terms_viz)

wordcloud = WordCloud(width=800, height=400, background_color='white', colormap

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Top Terms in Negative Sentiment Tweets')
plt.show()

print("Bar chart and Word Cloud generated successfully.")
```



Final Task

Subtask:

Summarize the key findings from the TF-IDF analysis of negative sentiment tweets, discussing the most prominent terms and their potential implications based on the visualizations.

Summary:

Q&A

The most prominent terms identified from the TF-IDF analysis of negative sentiment tweets are 'threat' (with a TF-IDF score of 29.2683), 'war' (27.4703), 'crime' (27.1579), 'suffer' (25.9910), and 'fear' (25.8400). These terms, along with others like 'attack', 'wrong', 'bad', 'worry', and 'trouble', indicate that negative sentiment in the analyzed tweets is primarily associated with discussions around conflict, violence, social issues, and emotional distress. The visualizations (bar chart and word cloud) confirm the high prominence of these terms, suggesting that users are expressing strong negative feelings in relation to these serious topics.

Data Analysis Key Findings

- TF-IDF vectorization successfully transformed 10,000 cleaned tweets into a matrix of (10000, 869) dimensions, representing 869 unique terms.
- The top 10 most prominent terms in negative sentiment tweets, based on their cumulative TF-IDF scores, are 'threat' (29.27), 'war' (27.47), 'crime' (27.16), 'suffer' (25.99), 'fear' (25.84), 'attack' (25.24), 'wrong' (24.83), 'bad' (23.93), 'worry' (23.55), and 'trouble' (22.51).
- Visualizations, including a bar chart and a word cloud, effectively highlighted these terms, reinforcing their significance within the negative sentiment discourse.

Insights or Next Steps

- Further analysis could involve exploring the context in which these top negative terms appear to understand the specific issues driving user dissatisfaction or concern.
- Consider grouping similar negative terms (e.g., 'war', 'attack', 'threat') into broader categories to identify overarching themes of negative sentiment, which could inform crisis management or policy responses.